PROJECT REPORT TITLE:
PPM Image Processing in Object-Oriented and Procedural Programming Paradigms

1. INTRODUCTION & MOTIVATION:

The task required the development of two separate implementations of a program that can represent, read, write, and modify Portable Pixmap (PPM) images in different programming paradigms, each in a different programming language. The programs needed to support the P3 (plain ASCII) variant of the Netpbm format. General-purpose abstractions such as a Pixel and Image structure, an array of image pixels, and image file type representations that derive their structure from Image were used to parse and validate file header information, construct a pixel representation of the data, and apply a set of transformations such as grayscale, color inversion, and horizontal mirroring.

This report describes the design and analysis of two separate solutions to this task: an object-oriented implementation in C++ and a procedural implementation in Python. The selection of C++ was driven by the project's emphasis on abstractions such as Pixel, Image, and format-specific image types derived from Image. These requirements naturally imply an object-oriented structure built on encapsulation, hierarchies, and behavior distributed across interacting objects. As noted by Sebesta, C++ is an imperative language with numerous object-oriented structures and mechanisms (class definitions, inheritance, and subtype polymorphism) that distribute control flow across different objects [1]. Thus, C++ was a suitable choice for designing a program that could directly map to the specification's abstractions. For the second approach, a procedural paradigm was selected to provide a structurally contrasting approach. Procedural programming organizes computation into a series of subroutines that explicitly consume and produce data [1]. This model aligns well with the basic operations required for PPM image processing (reading, transforming, and writing files), each of which can be expressed as a function operating on well-defined parameters. Python, though a multi-paradigm language, was chosen because it supports procedural programming without imposing other paradigms, allowing the second implementation to remain consistent with procedural design principles while also taking advantage of the language's streamlined syntax.

The remainder of this report is organized as follows. Section 2 details how each program works within its respective paradigm, emphasizing programming structures like abstractions. Section 3 evaluates the trade-offs between the object-oriented and procedural approaches, accounting for factors such as writability, resource performance, and adherence to the task specifications. Section 4 concludes with final observations regarding implementation outcomes and paradigm suitability.

2. DESCRIPTIONS OF IMPLEMENTATIONS

Both implementations developed for this project provide equivalent functionality for reading, representing, modifying, and writing to PPM files in P3 (ASCII) format. Each version includes generic read and write functions that operate on a file name and act as an interface through which specific image formats can be handled. The read operation decodes the file's metadata and pixel structure and encodes it into an internal pixel array that can be manipulated by image transformations. While reading file data, checks are continually performed to ensure the file is correctly formatted. If any structural issue is encountered (such as not enough pixels in a PPM file), the read operation is aborted and the error is

reported, though the specific mechanism for doing so differs between the two implementations, as detailed in Sections 2.1 and 2.2. With PPM files, read functions operate on the magic identifier, image dimensions, maximum color value, and triplets of RGB values. The generic write operation performs the inverse by serializing the pixel array representation into a valid image file. Three transformations are implemented for both versions: grayscale(), invert(), and horizontalflip() (abbreviated hflip()). The grayscale operation computes the average of the red, green, and blue components to replace the pixel with a uniform intensity value ((R + G + B) / 3) [2]. The inversion transformation applies a per-channel intensity negation by subtracting each component from the maximum color value [3]. Horizontal flipping mirrors the image by swapping pixel entries across each row using index arithmetic. PPM files may legally specify a maximum color value greater than 255 [4]. To support such images uniformly, both implementations normalize pixel values to the standard 8-bit range. A scalar factor (255 / maxColor) rescales each pixel component, ensuring that subsequent filters operate on consistent 0-255 values [5].

2.1 Object-Oriented Implementation in C++

The C++ solution follows an object-oriented design around an abstract Image class. This class encapsulates shared image attributes: width, height, the maximum color value, and an array of pixels stored in std::vector<Pixel>. This abstraction defines a common interface for image operations. Pixels are represented as a simple struct containing three integer RGB components. Because the underlying representation of the pixel array is a one-dimensional vector, the class provides a protected at(i,j) method that maps 2D image coordinates to the internal linear index. The Image class also declares virtual read() and write() methods, which enable derived classes to implement format-specific parsing and serialization. PPMImage, defined in PPMImage.h and implemented in PPMImage.cpp, overrides these methods to support the P3 PPM format. If malformed input is detected in any stage of the read method, the reset() method clears the object's internal state and reports the error via std::cerr. If there are no errors, a read will parse the header information, reserve space in the pixel vector for width * height elements, and copy sets of RGB triplets into the array by creating a temporary pixel object. If the file specifies a maxColor other than 255, the implementation performs 8-bit normalization by scaling each pixel using the previously mentioned factor. Because maxColor is inherited from Image, the resulting normalized value is stored directly in the object for all subsequent operations. Image transformations are member functions of Image and operate over the encapsulated PixelArray. Each transformation mutates the object's internal pixel data in place. State is encapsulated within the object, behavior is expressed by member functions, and inherited types such as PPMImage automatically gain access to these operations through polymorphism. The write() method outputs a properly formatted P3 header and serializes the pixel array in row-major order. Additional methods such as getWidth(), getHeight(), and reset() provide controlled access to internal state and support object reuse.

```
struct Pixel { int r, g, b; };
class Image {
public:
    virtual void read(const std::string& fileName) = 0;
    virtual void write(const std::string& fileName) = 0;
    void grayscale(); void invert(); void hflip();
protected:
    int width, height;
    std::vector<Pixel> PixelArray;
    Pixel& at(int i, int j) {return PixelArray.at((width * i) + j);} };
```

Figure 1: Excerpt of the Image header file demonstrating how the Image abstraction encapsulates shared state.

2.2 Procedural Implementation in Python

The Python implementation follows a procedural programming model where image data is represented using simple built-in data structures rather than object-oriented abstractions. In this design, an image is represented as a tuple (width, height, pixelArray), where pixelArray is a list of (r,g,b) integer triples. Because no state is encapsulated within objects, all operations explicitly receive an image tuple as input and return a new tuple with updated pixel data. Data flows through a sequence of stateless functions. To support extensibility, the implementation includes dictionaries named READERS and WRITERS, which map format identifiers to corresponding read and write functions. This construct provides a lightweight dispatch mechanism without requiring classes or inheritance. Reading PPM files is handled through the function readImgPPM(fileName), which parses the magic identifier, dimensions, and maximum color value directly from the file. Python's built-in iteration and list-processing features are used to collect pixel components effectively. A nested list comprehension flattens all remaining numeric tokens into a single list of integers, which is then grouped into RGB triples. Input validation is performed by raising exceptions when the header is incorrectly formatted or when the total number of pixel components is not divisible by three. There is no persistent object state, so errors terminate the read operation and return control to the caller. The same kind of normalization process is performed for maxColor, but as mentioned before, a new pixel list has to be returned when every pixel is manipulated. Only explicit data transformation is performed. Each image filter is expressed as an independent function that performs the same operations outlined before, with the results always appended to a new pixel list. For example, hflip() performs row-wise index manipulation by computing the mirrored position for each pixel and swapping the values in a copied list. All of these functions return a new image representation structured as a (width, height, pixelArray) tuple. The transformations are expressed as ordered, state-free computations rather than object methods. The write operation, implemented as writeImgPPM(), accepts a file name and image tuple, emits a P3 header, and serializes each pixel value sequentially. The file-handling model, "with open(...)", ensures resources are managed deterministically without auxiliary abstractions.

```
def grayscale(image):
    width, height, pixelArray = image
    grayArray = []
    for (r,g,b) in pixelArray:
        greyColor = (r + g + b) // 3
        grayArray.append((greyColor, greyColor, greyColor))
    return (width, height, grayArray)
```

Figure 2: The grayscale function in Python. It takes a raw image tuple and returns a new representation of it.

The structural differences between paradigms is evident when comparing how similar operations are expressed:

```
void Image::grayscale()
{
    int greyVal;
    for (auto& p : PixelArray)
    {
        greyVal = (p.r + p.g + p.b) / 3;
```

```
            p.r = p.g = p.b = greyVal;
        }
    }
```

Figure 3: The grayscale function in C++. It acts on the currently calling object and manipulates its pixel values.

2.3 Testing & Validation

To confirm the correctness of both implementations, a series of structured tests were performed using PPM files of varying sizes and color characteristics. These included a small synthetic image for verifying basic parsing, a large, realistic photograph to test performance on non-trivial solutions, and a small image with a non-255 maximum color value to validate normalization behavior. The C++ and Python programs executed the same test files. Smaller outputs were compared pixel-by-pixel, larger ones were viewed on the macro level to ensure their similarity. Below is an example of a test input/output using a large image.

Figure 4: This bird image is 2MB. From left to right: the original image, C++ inversion, and the Python inversion.

---

3. EVALUATION OF IMPLEMENTATIONS

The two implementations developed for this project differ because of the programming paradigm they use and the language they are written in. These differences are reflected in each implementation's maintainability, writability, and how well they satisfied the project's emphasis on meaningful abstractions. Although both solutions meet the functional requirements, the C++ implementation ultimately aligns more closely with the project's goals and provides a more scalable and semantically coherent foundation for image-processing tasks.

3.1 Maintainability & Scalability of Design

A primary point of divergence lies in how each paradigm organizes program structure for future adaptation. The C++ implementation uses an abstract base class (Image) and a format-specific subclass (PPMImage). Because this interface is defined in the base class and extended by derived types, new image formats such as PNGImage or BMPImage could be added by implementing format-specific read and write routines while inheriting shared behavior. This is exemplary of the object-oriented philosophy described by Sebesta, in which data abstraction, encapsulation, and inheritance enable programs to scale by adding new types that preserve the same conceptual interface [1]. The encapsulated state ensures that transformations operate consistently across all image variants using polymorphism. If additional filters

were introduced, they would require no structural changes beyond implementing additional member functions to Image. By contrast, the Python solution expresses all logic through loose collections of functions operating on raw tuples. While this procedural structure is readable and straightforward, extending the system would require manually coordinating multiple dictionaries, ensuring consistent tuple structure, and passing formats around explicitly. Nothing enforces a uniform interface across operations; each new feature depends on careful adherence to agreed-upon conventions. As Pierce emphasizes, the absence of static structure and type guarantees in untyped or loosely typed systems shifts the burden of correctness onto disciplined coding practices rather than the language's abstraction mechanisms [6]. This does not make the Python approach invalid, but it limits its ability to scale into a robust multi-format image-processing framework without eventually adopting class-like structures or additional layers of organization.

3.2 Readability & Writability

From a writability perspective, Python's procedural style provides very low syntactic overhead. Each transformation is expressed as a small function manipulating lists and tuples, using minimal boilerplate. According to Sebesta's language evaluation criteria, writability encompasses simplicity, orthogonality, and lack of verbosity [1]. Python performs strongly in these areas. For example, a grayscale transformation in Python can be implemented using a few expressive loops or comprehensions, and parsing text-based PPM files leverages built-in iteration constructs naturally. C++ requires substantially more structural setup: class declarations, headers, state initialization, and separation between interface and implementation. While this formality increases initial writing effort, it simultaneously enhances readability for larger systems because responsibilities are explicitly partitioned. A reader can quickly understand where shared behavior resides (Image) and where specialized logic begins (PPMImage). The explicitness of method signatures and the use of types make code navigation more predictable in larger programs. Thus, writability favors Python in small-scale tasks, but readability in long-lived or multi-developer systems favors C++ due to its richer structural cues and compilation-time guarantees.

3.3 Reliability & Adherence to Specification

Reliability, in the context of programming-language evaluation, relates to how effectively a language prevents or detects errors, enforces constraints, and supports predictable execution behavior [1]. The specification explicitly requires a "truthful abstraction" of images, format-specific types, and behavior distributed across structured abstractions. The C++ implementation offers reliability through several mechanisms. First, it offers type safety and explicit state representation, reducing the chance of misusing pixel data. Next, encapsulation guarantees that transformations operate only on a valid internal state. Then, virtual methods ensure each format-specific subclass obeys the same interface. Finally, error handling via reset() keeps partially read images in a consistent state. This reset-based model differs fundamentally from the Python implementation's fail-fast behavior. readImgPPM() raises an exception immediately upon encountering malformed input, halting execution at the point of failure, whereas the C++ read() method clears internal state and reports the error via std::cerr without interrupting program flow. This places the burden of checking the outcome on the caller, who must invoke isValid() after every read to confirm the object is usable; nothing in the class interface enforces this check at compile time. In this respect, C++'s reliability advantage is conditional rather than absolute: robustness still depends on disciplined caller behavior, though the presence of an explicit isValid() method at least makes the check available and self-documenting in a way Python's convention-based error propagation does not. These

properties align with Pierce's description of how type systems and abstractions prevent erroneous behaviors by structurally restricting the space of possible programs [6]. The procedural Python implementation also performs thorough error checking through exceptions, but it cannot enforce structural constraints beyond what the programmer manually verifies. Because image data is represented by general-purpose tuples and lists, there is no guarantee that a malformed tuple will not be accidentally passed to a filter. Maintaining consistency relies solely on programmer discipline and runtime checks, which makes reliability more reactive than proactive. Overall, the C++ solution more closely matches and is more suitable for the project's emphasis on abstractions. Even though the structure could be reflected in other paradigms like procedural, and the Python version captures the functional requirements, C++ better fits the required abstraction specifications.

---

4. CONCLUSIONS

This project required implementing a complete PPM (P3) image-processing system in two different programming paradigms, each using a different language. Both implementations were able to read and validate PPM headers, construct a pixel representation, apply filter transformations, normalize non-255 color ranges, and output fully compliant image files. In C++, this functionality was organized around a hierarchy of abstractions (Pixel, Image, and PPMImage), where each class encapsulated behavior and state relevant to image manipulation. In Python, the same operations were expressed procedurally using simple tuples and functions that transformed and returned new image data. Despite their differences, both approaches successfully met the project's functional goals and demonstrated how equivalent tasks can be mapped to distinct programming paradigms.

While different paradigms can be implemented to meet the required behavior, C++ ultimately aligned more closely with the project's emphasis on abstractions and type-structured design. The class hierarchy provided a clean separation of responsibilities and allowed format-specific functionality to extend a general interface naturally. Meanwhile, Python's procedural implementation excelled in clarity and minimalism but would require additional structural mechanisms to scale beyond the project's basic requirements. Thus, the suitability of a paradigm depends heavily on the nature and long-term goals of a task. Problems involving layered abstractions, strong invariants, or multiple related data types tend to benefit more from object-oriented structures, where encapsulation and polymorphism express relationships explicitly. Tasks that emphasize data transformation, scripting, or linear pipelines often map more naturally to procedural or functional approaches, where structural depth is not as important as simplicity and directness. A small example of this outside the two paradigm implementations is the project's run.py utility, which compiles the C++ solution when its sources have changed and launches either implementation. As a short, linear orchestration task, it was naturally suited to a procedural scripting approach. Understanding how different paradigms frame a problem, what extensibility, reliability, and design clarity they offer, and how they influence the expression of abstractions is ultimately essential for selecting the most effective tools and approaches in future development contexts.

REFERENCES

[1] Robert W. Sebesta, *Concepts of Programming Languages,* 12th ed., Pearson, 2019.
[2] *Stanford CS101 - Image-6 Grayscale.* Stanford University, Available at: https://web.stanford.edu/class/cs101/image-6-grayscale-adva.html (Accessed Nov. 27, 2025).
[3] GeeksforGeeks, "Python - Color Inversion using Pillow", last updated April 28, 2025. Available at: https://www.geeksforgeeks.org/python/python-color-inversion-using-pillow/ (Accessed Nov. 27, 2025).
[4] *PPM Format Specification*. Netpbm. Last updated Nov. 7, 2025. Available at: https://netpbm.sourceforge.net/doc/ppm.html (Accessed Nov. 27, 2025).
[5] Rafael C. Gonzalez and Richard E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.
[6] Benjamin C. Pierce, *Types and Programming Languages.* MIT Press, 2002.